

DAY 4

PostgreSQL 18 Backup & Disaster Recovery

Step-by-Step Lab Manual for a LIVE Patroni / Zalando Postgres Operator Cluster

Namespace	postgres-ha
Cluster	acid-postgres-ha (Zalando Operator + Patroni)
PostgreSQL	18
Backup tools	pg_dump, pg_dumpall, pg_basebackup, WAL archiving, pgBackRest
Enterprise backup	Full / Incremental / Differential / Encryption / Retention / PITR
Restore rule	Restore only to a separate recovery cluster or replacement PVCs

CRITICAL SAFETY RULE

Backups are taken while acid-postgres-ha is live. Restore, full recovery and PITR exercises must NOT overwrite the active Patroni member PVCs. A restore changes PGDATA and must be performed on isolated recovery storage or a separate recovery cluster. Patroni failover is not a substitute for backup.

Covers every requested Day 4 topic and lab exercise

Table of Contents

1. 0. Lab Architecture, Scope and Safety
2. 1. Pre-Lab Validation of the Live Patroni Cluster
3. 2. Backup Concepts: Logical, Physical, WAL and PITR
4. 3. Lab 1 - Logical Backup from the Live Cluster
5. 4. Lab 2 - Physical Backup with pg_basebackup
6. 5. Lab 3 - WAL Archiving Validation
7. 6. Lab 4 - Configure pgBackRest for the Live Cluster
8. 7. Lab 5 - Full Backup with pgBackRest
9. 8. Lab 6 - Incremental Backup
10. 9. Lab 7 - Differential Backup
11. 10. Repository, Compression, Encryption and Retention
12. 11. Lab 8 - Restore Database / Full Cluster Recovery
13. 12. Lab 9 - Point-in-Time Recovery (PITR)
14. 13. Patroni Failover Considerations During Backup
15. 14. Validation, Monitoring, Troubleshooting and Cleanup
16. 15. RPO / RTO and Backup Schedule
17. 16. Final Coverage Checklist and References

Implementation note

The existing Zalando/Spilo stack commonly uses WAL-G/WAL-E for native physical backups. pgBackRest is not automatically present in the default Spilo image. The pgBackRest part of this manual therefore uses a custom Spilo image that contains pgBackRest and a shared/object repository. Do not apt-install packages into a running production pod and assume the change will survive a restart.

0. Lab Architecture, Scope and Safety

0.1 What runs live and what is isolated?

- **LIVE:** acid-postgres-ha continues to serve traffic and remains Patroni-managed during backup exercises.
- **LIVE BACKUP:** logical dumps, pg_basebackup, WAL archive checks and pgBackRest backup commands are performed against the current primary or an eligible replica.
- **ISOLATED RESTORE:** destructive restore/PITR is performed into a recovery cluster/PVC. Never replace files below the active primary PGDATA while Patroni is running.
- **REPOSITORY:** use a durable pgBackRest repository. On Kubernetes, object storage such as GCS/S3 is preferred. An RWX PVC can be used for training if every member can mount it.

0.2 Recovery architecture

Application -> acid-postgres-ha Service -> Patroni Primary

```

|
+--> Logical backup (pg_dump / pg_dumpall)
+--> Physical backup (pg_basebackup)
+--> WAL archive stream
+--> pgBackRest repository (GCS/S3 or RWX)

```

Recovery path:

pgBackRest repository + archived WAL -> isolated recovery PGDATA -> validation -> controlled cutover

0.3 Why failover is not backup

Patroni protects availability by promoting a replica when the primary fails. It does not protect against logical mistakes such as DROP TABLE, accidental DELETE, bad application updates, corruption copied to replicas, or retention mistakes. Backups and PITR protect against those events.

0.4 Required access and tools

- kubectl access to namespace postgres-ha
- Permission to read/patch the postgresql CR and inspect StatefulSets/PVCs
- PostgreSQL superuser credentials available through the existing cluster secret or local socket
- Container registry access if you build a custom Spilo image with pgBackRest
- A backup repository: GCS/S3 strongly preferred; otherwise an RWX PersistentVolumeClaim
- jq is useful but optional

1. Pre-Lab Validation of the Live Patroni Cluster

1.1 Set namespace and list resources

```
kubectl config set-context --current --namespace=postgres-ha
kubectl get pods -o wide
kubectl get svc
kubectl get postgresql
kubectl get pvc
kubectl get statefulset
```

1.2 Identify the current Patroni primary - never assume pod-0

```
for p in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do
  echo "=== $p ==="
  kubectl exec "$p" -- curl -s http://127.0.0.1:8008/ | jq -r '"role=" + (.role // "unknown") + " state=" + (.state // "unknown")'
done
```

Expected

Exactly one member should report primary/leader and the others should report replica/standby. If the cluster is not healthy, stop the backup lab and repair HA first.

1.3 Set shell variables

```
NS=postgres-ha
CLUSTER=acid-postgres-ha
PRIMARY=acid-postgres-ha-0    # CHANGE to the actual primary
REPLICA=acid-postgres-ha-1    # choose a healthy replica

kubectl -n $NS exec $PRIMARY -- psql -U postgres -Atqc "select pg_is_in_recovery(), current_database(), version();"

```

1.4 Discover the actual PGDATA path

```
kubectl -n $NS exec $PRIMARY -- psql -U postgres -Atqc "show data_directory;"
kubectl -n $NS exec $PRIMARY -- sh -c 'echo "PGDATA=$PGDATA"; ls -ld "$PGDATA" 2>/dev/null || true'
```

Do not hardcode PGDATA

Spilo commonly uses a path below /home/postgres/pgdata, but the authoritative value is SHOW data_directory. Use the value returned by PostgreSQL in every restore plan.

1.5 Capture a baseline

```
kubectl -n $NS exec $PRIMARY -- psql -U postgres -c "select now(), pg_current_wal_lsn(), pg_is_in_recovery();"

kubectl -n $NS exec $PRIMARY -- psql -U postgres -c "select application_name, client_addr, state, sync_state,
    sent_lsn, write_lsn, flush_lsn, replay_lsn
from pg_stat_replication;"
```

2. Backup Concepts: Logical, Physical, WAL and PITR

Method	Backs up	Can restore one DB/table?	Supports PITR?	Typical use
pg_dump / pg_dumpall	SQL objects and data	Yes	No	Migration, object/database recovery
pg_basebackup	Entire physical cluster	No	Yes, with WAL archive	Standby/base backup, DR
WAL archiving	Continuous change records	No	Enables PITR	Continuous recovery stream
pgBackRest	Physical cluster + WAL metadata	No	Yes	Enterprise backup/restore/PITR

2.1 Full vs differential vs incremental

- Full: copies the complete PostgreSQL cluster and is independently restorable.
- Differential: copies files changed since the last full backup. Restore requires the differential plus its full backup.
- Incremental: copies files changed since the immediately preceding backup. Restore may depend on a chain of incremental/differential/full backups.
- Archived WAL is still required to roll forward from the selected backup to a later recovery point.

2.2 RPO and RTO

- RPO (Recovery Point Objective): acceptable amount of data loss. Frequent WAL archiving reduces potential data loss.
- RTO (Recovery Time Objective): acceptable time to restore service. Backup size, network speed, WAL replay volume and recovery validation all affect RTO.

3. Lab 1 - Logical Backup from the Live Cluster

3.1 Full cluster logical backup

```
mkdir -p backups/logical

kubectl -n $NS exec $PRIMARY -- pg_dumpall -U postgres --clean --if-exists > backups/logical/${CLUSTER}-dumpall-$(date +%Y%m%d-%H%M%S).sql

ls -lh backups/logical/
```

3.2 Recommended custom-format backup of the banking database

```
kubectl -n $NS exec $PRIMARY -- pg_dump -U postgres -Fc -d banking -f /tmp/banking.dump

kubectl -n $NS cp $PRIMARY:/tmp/banking.dump backups/logical/banking-$(date +%Y%m%d-%H%M%S).dump
```

3.3 Verify the dump without restoring

```
pg_restore -l backups/logical/banking-*.dump | head -40

# For a SQL dump:

head -30 backups/logical/${CLUSTER}-dumpall-*.sql
```

3.4 Test logical restore into an isolated database

```
kubectl -n "$NS" exec "$PRIMARY" -- \
    psql -U postgres -c "DROP DATABASE IF EXISTS banking_restore_test;"

kubectl -n "$NS" exec "$PRIMARY" -- \
    psql -U postgres -c "CREATE DATABASE banking_restore_test;"

BACKUP_FILE=$(ls -lt backups/logical/banking-*.dump | head -1)
kubectl -n "$NS" cp "$BACKUP_FILE" \
    "$PRIMARY:/tmp/banking-restore.dump"

kubectl -n "$NS" exec "$PRIMARY" -- \
    pg_restore -U postgres \
    -d banking_restore_test \
    --clean --if-exists \
    /tmp/banking-restore.dump
```

Live-cluster caution

Creating a temporary restore database consumes I/O and storage on the production primary. For a real production system, restore-test into a dedicated recovery cluster instead. The command above is appropriate only for a controlled training cluster.

4. Lab 2 - Physical Backup with pg_basebackup

4.1 Why use a replica when possible?

A physical backup can be taken while PostgreSQL is online. Running it from a healthy replica can reduce backup read I/O on the primary, but verify that the replica is caught up and that your replication/authentication rules permit the backup connection.

4.2 Inspect replication status

```
kubectl -n $NS exec $PRIMARY -- psql -U postgres -c "select application_name, state, sync_state,
pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)) as lag
from pg_stat_replication;"
```

4.3 Create a compressed tar-format base backup in the pod

```
kubectl -n $NS exec $REPLICA -- sh -lc '
rm -rf /tmp/basebackup && mkdir -p /tmp/basebackup &&
pg_basebackup -h 127.0.0.1 -p 5432 -U postgres -D /tmp/basebackup -Ft -z -Xs -P
'
```

Authentication

If the local PostgreSQL connection requires a password or rejects replication protocol access, use the cluster replication user/secret rather than weakening pg_hba.conf. Do not add broad trust rules for the lab.

4.4 Copy the backup out and verify files

```
mkdir -p backups/physical
kubectl -n $NS cp $REPLICA:/tmp/basebackup ./backups/physical/basebackup-$(date +%Y%m%d-%H%M%S)
find backups/physical -maxdepth 2 -type f -ls | head
```

4.5 Important limitation

A base backup is a cluster-level physical backup. It cannot restore a single database independently. To recover to a specific time, the required archived WAL sequence must also be available.

5. Lab 3 - WAL Archiving Validation

5.1 Inspect current live-cluster archive settings

```
kubectl -n $NS exec $PRIMARY -- psql -U postgres -c "show archive_mode;"
kubectl -n $NS exec $PRIMARY -- psql -U postgres -c "show archive_command;"
kubectl -n $NS exec $PRIMARY -- psql -U postgres -c "show archive_library;"
kubectl -n $NS exec $PRIMARY -- psql -U postgres -x -c "select * from pg_stat_archiver;"
```

5.2 Force a WAL switch and check archive progress

```
BEFORE=$(kubectl -n $NS exec $PRIMARY -- psql -U postgres -Atqc "select archived_count from
pg_stat_archiver;")
kubectl -n $NS exec $PRIMARY -- psql -U postgres -c "select pg_switch_wal();"
sleep 10
kubectl -n $NS exec $PRIMARY -- psql -U postgres -c "select archived_count, failed_count, last_archived_wal,
last_archived_time,
        last_failed_wal, last_failed_time
        from pg_stat_archiver;"
echo "Archived before test: $BEFORE"
```

5.3 Check native Zalando/Spilo backup tooling

```
kubectl -n $NS exec $PRIMARY -- sh -lc '
command -v wal-g || true
command -v wal-e || true
env | grep -E "(USE_WALG|WALG_|WALE_)" | sed -E "s/(SECRET|KEY|TOKEN|PASSWORD)=.*/\1=REDACTED/"
'
```

Do not run two archive_command implementations at once

A Spilo cluster may already archive WAL with WAL-G/WAL-E. Replacing archive_command with pgBackRest without planning can break the operator backup path. During migration, choose one authoritative WAL archiver and test it before relying on PITR.

6. Lab 4 - Configure pgBackRest for the Live Cluster

6.1 Design used by this lab

- Build a custom Spilo image that is identical to the current image plus pgBackRest.
- Use an object-storage pgBackRest repository (GCS example below). This avoids ReadWriteOnce PVC problems across three Patroni pods.
- Mount pgBackRest configuration and credentials as Kubernetes Secret/ConfigMap.
- Patch the PostgreSQL CR to use the custom image and required volumes in a controlled maintenance window.
- Configure Patroni/PostgreSQL archive_command to call pgBackRest only after validating the repository and after deciding how to coexist with/replace WAL-G.

6.2 Record current live image and configuration

```
CURRENT_IMAGE=$(kubectl -n $NS get pod $PRIMARY -o jsonpath='{.spec.containers[?(@.name=="postgres")].image}')
echo "$CURRENT_IMAGE"

kubectl -n $NS get postgresql $CLUSTER -o yaml > ${CLUSTER}-before-pgbackrest.yaml
kubectl -n $NS get statefulset $CLUSTER -o yaml > ${CLUSTER}-sts-before-pgbackrest.yaml
```

6.3 Build a custom Spilo image that contains pgBackRest

```
cat > Dockerfile.spilo-pgbackrest <<'EOF'
ARG BASE_IMAGE
FROM ${BASE_IMAGE}
USER root
RUN apt-get update \
    && apt-get install -y --no-install-recommends pgbackrest \
    && rm -rf /var/lib/apt/lists/*
USER postgres
EOF

# Replace registry.example.com with your registry
CUSTOM_IMAGE=registry.example.com/spilo-pgbackrest:pg18

docker build --build-arg BASE_IMAGE="$CURRENT_IMAGE" -f Dockerfile.spilo-pgbackrest -t "$CUSTOM_IMAGE" .
docker push "$CUSTOM_IMAGE"
```

If the current Spilo base image does not use apt

Inspect /etc/os-release and the image build process, then install a pgBackRest package compatible with that OS and PostgreSQL 18. The important requirement is that every Patroni member uses the same immutable image and pgBackRest version.

6.4 Create repository encryption secret

```
PGBR_CIPHER_PASS=$(openssl rand -base64 48)

kubectl -n $NS create secret generic pgbackrest-repo-secret --from-literal=repo-cipher-
pass="$PGBR_CIPHER_PASS"

unset PGBR_CIPHER_PASS
```

6.5 GCS repository prerequisites

- Create a dedicated GCS bucket for pgBackRest backups.
- Grant the pod identity/service account least-privilege access to that bucket.
- Prefer Workload Identity and repo1-gcs-key-type=auto. If you must use a service-account JSON key, mount it from a Kubernetes Secret and rotate it.

6.6 Create pgBackRest configuration

```
cat > pgbackrest.conf <<'EOF'

[postgres-ha]
pg1-path=/home/postgres/pgdata/pgroot/data
pg1-port=5432

[global]
repo1-type=gcs
repo1-gcs-bucket=REPLACE_WITH_BUCKET_NAME
repo1-gcs-key-type=auto
repo1-path=/pgbackrest/postgres-ha
repo1-retention-full=2
repo1-retention-diff=4
repo1-cipher-type=aes-256-cbc
compress-type=zst
compress-level=3
process-max=2
start-fast=y
log-level-console=info

[global:archive-push]
process-max=2

[global:archive-get]
process-max=2
EOF

kubectl -n $NS create configmap pgbackrest-config --from-file=pgbackrest.conf=./pgbackrest.conf
```

PGDATA path check

Replace pg1-path with the exact value returned by SHOW data_directory if your Spilo layout differs.

6.7 Patch the cluster to use the custom image and mount config/secret

```
kubectl -n $NS edit postgresql $CLUSTER
```

```
# Add/merge these fields under spec:
# dockerImage: registry.example.com/spilo-pgbackrest:pg18
# additionalVolumes:
# - name: pgbackrest-config
#   mountPath: /etc/pgbackrest
#   targetContainers:
#     - postgres
#   volumeSource:
#     configMap:
#       name: pgbackrest-config
# - name: pgbackrest-secret
#   mountPath: /etc/pgbackrest-secret
#   targetContainers:
#     - postgres
#   volumeSource:
#     secret:
#       secretName: pgbackrest-repo-secret
```

Controlled restart expected

Changing dockerImage causes database pods to be recreated. Patroni should maintain service by rolling members safely, but do this during a maintenance window and monitor the primary role and replication after every pod replacement.

6.8 Validate pgBackRest on every member

```
for p in $(kubectl -n $NS get pod -l cluster-name=$CLUSTER -o name | cut -d/ -f2); do
    echo "=== $p ==="
    kubectl -n $NS exec $p -- pgbackrest version
    kubectl -n $NS exec $p -- test -r /etc/pgbackrest/pgbackrest.conf && echo config-ok
done
```

6.9 Expose the cipher pass to pgBackRest

Do not place the passphrase in the ConfigMap. Provide it to the pgBackRest command from the mounted Secret. One simple training pattern is to export PGBACKREST_REPO1_CIPHER_PASS immediately before each pgBackRest command:

```
kubectl -n $NS exec $PRIMARY -- sh -lc '
    export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"
    pgbackrest --stanza=postgres-ha info
'
```

6.10 Create the stanza on the current primary

```
kubectl -n $NS exec $PRIMARY -- sh -lc '
    export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"
    pgbackrest --stanza=postgres-ha stanza-create
    pgbackrest --stanza=postgres-ha check
'
```

Before changing archive_command

stanza-create/check must succeed and repository access must be confirmed. If the live cluster is already archiving with WAL-G, document the migration/cutover plan before switching WAL archiving to pgBackRest.

6.11 Configure WAL archiving through Patroni/PostgreSQL

```
# First inspect Patroni dynamic config
kubectl -n $NS exec $PRIMARY -- patronictl list
kubectl -n $NS exec $PRIMARY -- patronictl show-config

# Edit the postgresql.parameters section through Patroni
kubectl -n $NS exec -it $PRIMARY -- patronictl edit-config

# Set:
# archive_mode: 'on'
# archive_command: >-
#   PGBACKREST_REPO1_CIPHER_PASS=$(cat /etc/pgbackrest-secret/repo-cipher-pass)
#   pgbackrest --stanza=postgres-ha archive-push %p
```

Patroni rule

For Patroni-managed parameters, avoid hand-editing postgresql.conf inside the pod. Use the operator CR or Patroni dynamic configuration so the setting survives restart/failover.

7. Lab 5 - Full Backup with pgBackRest

7.1 Verify archive health first

```
kubectl -n $NS exec $PRIMARY -- psql -U postgres -c "select pg_switch_wal();"
sleep 10
kubectl -n $NS exec $PRIMARY -- psql -U postgres -c "select archived_count, failed_count, last_archived_wal,
last_archived_time from pg_stat_archiver;"
```

7.2 Run full backup

```
kubectl -n $NS exec $PRIMARY -- sh -lc '
export PGBACKREST_REP01_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"
pgbackrest --stanza=postgres-ha --type=full backup
'
```

7.3 Inspect backup metadata

```
kubectl -n $NS exec $PRIMARY -- sh -lc '
export PGBACKREST_REP01_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"
pgbackrest --stanza=postgres-ha info
'
```

Expected

The backup set label ends with F. Record the start/stop times, database size, repository size and the WAL start/stop values.

7.4 Create test data after full backup

```
kubectl -n $NS exec $PRIMARY -- psql -U postgres -d banking -c "create table if not exists dr_lab_events(
  id bigint generated always as identity primary key,
  event_time timestamptz default clock_timestamp(),
  note text);
insert into dr_lab_events(note) values ('after full backup');
select * from dr_lab_events order by id;"
```

8. Lab 6 - Incremental Backup

8.1 Add more changed data

```
kubectl -n $NS exec $PRIMARY -- psql -U postgres -d banking -c "insert into dr_lab_events(note) values ('before incremental backup');"
```

8.2 Run incremental

```
kubectl -n $NS exec $PRIMARY -- sh -lc '
  export PGBACKREST_REP01_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"
  pgbackrest --stanza=postgres-ha --type=incr backup
  pgbackrest --stanza=postgres-ha info
'
```

Expected

The new label ends with I and references an earlier backup set. Incremental backups are normally smaller but have a longer dependency chain.

8.3 Validate backup chain

```
kubectl -n $NS exec $PRIMARY -- sh -lc '
  export PGBACKREST_REP01_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"
  pgbackrest --stanza=postgres-ha info --output=json
' | jq .
```

9. Lab 7 - Differential Backup

9.1 Make another change

```
kubectl -n $NS exec $PRIMARY -- psql -U postgres -d banking -c "insert into dr_lab_events(note) values ('before differential backup');"
```

9.2 Run differential

```
kubectl -n $NS exec $PRIMARY -- sh -lc '
export PGBACKREST_REP01_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"
pgbackrest --stanza=postgres-ha --type=diff backup
pgbackrest --stanza=postgres-ha info
'
```

Expected

The label ends with D. The differential backup depends on the most recent full backup, not on the latest incremental backup.

9.3 Compare backup types

Type	Source baseline	Restore dependencies	Use case
Full	None	Only full + required WAL	Weekly/base anchor
Differential	Last full	Full + chosen diff + WAL	Fast restore with reduced size
Incremental	Previous backup	Full/diff + incrementals + WAL	Frequent low-volume backup

10. Repository, Compression, Encryption and Retention

10.1 Repository

The repository is the durable storage containing pgBackRest backup sets, metadata and archived WAL. In this manual the primary example is a GCS bucket because it is accessible from all Patroni members and remains independent of a single Kubernetes node.

10.2 Compression

```
# Current example
compress-type=zst
compress-level=3
```

Compression reduces repository size and network transfer. Higher compression can increase CPU cost. Measure backup duration and CPU before increasing the level.

10.3 Encryption

```
repo1-cipher-type=aes-256-cbc
# Passphrase is supplied from Secret as PGBACKREST_REPO1_CIPHER_PASS
```

Encryption lifecycle

Choose repository encryption before initializing the repository/stanza and protect the cipher passphrase like a database credential. If you lose it, encrypted backups cannot be restored.

10.4 Retention policy

```
repo1-retention-full=2
repo1-retention-diff=4
```

When older backups expire, dependent differential/incremental sets are removed according to pgBackRest retention rules. WAL must not be expired more aggressively than your required PITR window.

10.5 Dry-run expiration before deleting anything

```
kubectl -n $NS exec $PRIMARY -- sh -lc '
  export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"
  pgbackrest --stanza=postgres-ha --dry-run expire
'
```

10.6 Run expiration after review

```
kubectl -n $NS exec $PRIMARY -- sh -lc '
  export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"
  pgbackrest --stanza=postgres-ha expire
'
```

11. Lab 8 - Restore Database / Full Cluster Recovery

This is destructive at PGDATA level

Do not perform the following restore against acid-postgres-ha live PVCs. Create a recovery cluster/PVC first. The live cluster remains the backup source; the recovery target is isolated.

11.1 Create a separate recovery PostgreSQL CR skeleton

```
cat > acid-pgbackrest-recovery.yaml <<'EOF'
apiVersion: acid.zalan.do/v1
kind: postgresql
metadata:
  name: acid-pgbackrest-recovery
  namespace: postgres-ha
spec:
  teamId: acid
  numberOfInstances: 1
  volume:
    size: 20Gi
  postgresql:
    version: "18"
    dockerImage: registry.example.com/spilo-pgbackrest:pg18
EOF
kubectl apply -f acid-pgbackrest-recovery.yaml
```

11.2 Stop recovery PostgreSQL before physical restore

A pgBackRest physical restore requires the target PostgreSQL server to be stopped. With an operator-managed recovery cluster, the cleanest training approach is to scale its StatefulSet to zero, mount its PVC in a toolbox pod, restore PGDATA, then let the operator start the pod again.

```
REC_STS=acid-pgbackrest-recovery
kubectl -n $NS scale statefulset/$REC_STS --replicas=0
kubectl -n $NS get pvc | grep $REC_STS
```

11.3 Create a recovery toolbox pod

```
# First find the recovery data PVC name:
REC_PVC=$(kubectl -n $NS get pvc -o name | grep acid-pgbackrest-recovery | head -1 | cut -d/ -f2)
echo "$REC_PVC"

cat > pgbackrest-recovery-toolbox.yaml <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: pgbackrest-recovery-toolbox
  namespace: $NS
spec:
  restartPolicy: Never
  containers:
```

```

- name: toolbox
  image: registry.example.com/spilo-pgbackrest:pg18
  command: ["sh", "-lc", "sleep infinity"]
  volumeMounts:
    - name: data
      mountPath: /home/postgres/pgdata
    - name: config
      mountPath: /etc/pgbackrest
    - name: secret
      mountPath: /etc/pgbackrest-secret
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: $REC_PVC
    - name: config
      configMap:
        name: pgbackrest-config
    - name: secret
      secret:
        secretName: pgbackrest-repo-secret

```

EOF

```
kubectl apply -f pgbackrest-recovery-toolbox.yaml
```

```
kubectl -n $NS wait --for=condition=Ready pod/pgbackrest-recovery-toolbox --timeout=10m
```

11.4 Discover and prepare recovery PGDATA

```
kubectl -n $NS exec pgbackrest-recovery-toolbox -- sh -lc '
ls -la /home/postgres/pgdata
find /home/postgres/pgdata -maxdepth 3 -name PG_VERSION -print
'
```

Set REC_PGDATA to the actual cluster data directory on the recovery PVC. Back up or move the freshly initialized recovery data directory before restoring.

```
REC_PGDATA=/home/postgres/pgdata/pgroot/data # verify before use
kubectl -n $NS exec pgbackrest-recovery-toolbox -- sh -lc "mv $REC_PGDATA ${REC_PGDATA}.before-restore &&
mkdir -p $REC_PGDATA"
```

11.5 Restore latest backup

```
kubectl -n $NS exec pgbackrest-recovery-toolbox -- sh -lc '
export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"
pgbackrest --stanza=postgres-ha --pg1-path=/home/postgres/pgdata/pgroot/data --set=latest --
type=default --target-action=promote restore
'
```

11.6 Remove toolbox and start the recovery cluster

```
kubectl -n $NS delete pod pgbackrest-recovery-toolbox --wait=true
kubectl -n $NS scale statefulset/$REC_STS --replicas=1
kubectl -n $NS rollout status statefulset/$REC_STS --timeout=10m
```

11.7 Validate restored cluster

```
REC_POD=$(kubectl -n $NS get pod -l cluster-name=acid-pgbackrest-recovery -o
jsonpath='{.items[0].metadata.name}')

kubectl -n $NS exec $REC_POD -- psql -U postgres -Atqc "select pg_is_in_recovery(), now();"
kubectl -n $NS exec $REC_POD -- psql -U postgres -d banking -c "select * from dr_lab_events order by id;"
```

12. Lab 9 - Point-in-Time Recovery (PITR)

12.1 Take a fresh full backup before the incident simulation

```
kubectl -n $NS exec $PRIMARY -- sh -lc '
  export PGBACKREST_REP01_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"
  pgbackrest --stanza=postgres-ha --type=full backup
  pgbackrest --stanza=postgres-ha info
'
```

12.2 Create a known-good recovery point

```
kubectl -n $NS exec $PRIMARY -- psql -U postgres -d banking -c "insert into dr_lab_events(note) values ('PITR
good row');"

TARGET_TIME=$(kubectl -n $NS exec $PRIMARY -- psql -U postgres -Atqc "select to_char(clock_timestamp() at time
zone 'UTC',
                                'YYYY-MM-DD HH24:MI:SS.US') || '+00';")
echo "PITR TARGET = $TARGET_TIME"
sleep 3
```

12.3 Simulate an accidental data loss event

```
kubectl -n $NS exec $PRIMARY -- psql -U postgres -d banking -c "delete from dr_lab_events;"
kubectl -n $NS exec $PRIMARY -- psql -U postgres -d banking -c "select count(*) from dr_lab_events;"
kubectl -n $NS exec $PRIMARY -- psql -U postgres -c "select pg_switch_wal();"
```

Record the timestamp externally

Do not keep the only copy of TARGET_TIME in the pod. Copy it into your lab notes or shell history. Recovery targets must be unambiguous and use timezone-aware timestamps.

12.4 Stop and prepare the isolated recovery target

```
kubectl -n $NS scale statefulset/$REC_STS --replicas=0
kubectl -n $NS wait --for=delete pod/$REC_POD --timeout=180s || true
kubectl apply -f pgbackrest-recovery-toolbox.yaml
kubectl -n $NS wait --for=condition=Ready pod/pgbackrest-recovery-toolbox --timeout=10m
```

12.5 Restore to the target time

```
# Ensure TARGET_TIME is set in your local shell before this command.
# Prepare empty PGDATA on the recovery PVC as in Lab 8, then:

kubectl -n $NS exec pgbackrest-recovery-toolbox -- sh -lc '
    export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"
    pgbackrest --stanza=postgres-ha --pg1-path=/home/postgres/pgdata/pgroot/data --type=time --
    target="REPLACE_WITH_TARGET_TIME" --target-action=promote restore
'
```

12.6 Start and validate PITR result

```
kubectl -n $NS delete pod pgbackrest-recovery-toolbox --wait=true
kubectl -n $NS scale statefulset/$REC_STS --replicas=1
kubectl -n $NS rollout status statefulset/$REC_STS --timeout=10m

REC_POD=$(kubectl -n $NS get pod -l cluster-name=acid-pgbackrest-recovery -o
jsonpath='{.items[0].metadata.name}')

kubectl -n $NS exec $REC_POD -- psql -U postgres -d banking -c "select * from dr_lab_events order by id;"
kubectl -n $NS exec $REC_POD -- psql -U postgres -c "select pg_is_in_recovery();"
kubectl -n $NS exec $REC_POD -- psql -U postgres -c "select timeline_id from pg_control_checkpoint();"
```

Expected PITR result

Rows committed before TARGET_TIME are present. The accidental DELETE is absent. pg_is_in_recovery() is false after promote. Recovery creates/uses a new timeline branch.

13. Patroni Failover Considerations During Backup

13.1 What if primary changes during the lab?

- Always rediscover PRIMARY before each backup or archive operation.
- Do not keep a stale PRIMARY variable after a Patroni switchover/failover.
- When archive_command is configured through Patroni, every member inherits the configuration and the promoted member can continue archiving.
- Keep the repository reachable from every potential primary.
- After failover, run pgBackRest check and force a WAL switch on the new primary before trusting PITR continuity.

13.2 Re-discover primary helper

```
PRIMARY=$(for p in $(kubectl -n $NS get pod -l cluster-name=$CLUSTER -o name | cut -d/ -f2); do
  role=$(kubectl -n $NS exec $p -- curl -s http://127.0.0.1:8008/ | jq -r '.role')
  [ "$role" = "primary" ] || [ "$role" = "master" ] || [ "$role" = "leader" ] && echo $p
done | head -1)
echo "Current primary: $PRIMARY"
```

13.3 Post-failover backup validation

```
kubectl -n $NS exec $PRIMARY -- psql -U postgres -c "select pg_switch_wal();"
kubectl -n $NS exec $PRIMARY -- sh -lc '
  export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"
  pgbackrest --stanza=postgres-ha check
  pgbackrest --stanza=postgres-ha info
'
```

14. Validation, Monitoring, Troubleshooting and Cleanup

14.1 Daily validation commands

```
# PostgreSQL archiver
kubectl -n $NS exec $PRIMARY -- psql -U postgres -x -c "select * from pg_stat_archiver;"

# pgBackRest repository / backup metadata
kubectl -n $NS exec $PRIMARY -- sh -lc '
    export PGBACKREST_REP01_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"
    pgbackrest --stanza=postgres-ha check
    pgbackrest --stanza=postgres-ha info
'

# Patroni health
kubectl -n $NS exec $PRIMARY -- patronictl list
```

14.2 Common failures

Symptom	Likely cause	Action
archive-push fails	Repository auth/network/cipher config	Run pgbackrest check; verify bucket IAM, DNS and Secret mount
stanza-create cannot find PostgreSQL	Wrong pg1-path / permissions	Use SHOW data_directory and verify uid/permissions
backup aborts after failover	Command targeted old primary / archive gap	Rediscover primary; confirm repository continuity and archive status
PITR stops too early	Missing archived WAL	Check archive errors and retention; restore from older valid backup if needed
Recovery pod cannot mount PVC	RWO volume still attached	Scale recovery StatefulSet to zero and wait for old pod deletion
pgBackRest version missing after restart	Package installed manually in pod	Use immutable custom Spilo image; do not apt-install ad hoc

14.3 Cleanup only the lab recovery objects

```
kubectl -n $NS delete postgresql acid-pgbackrest-recovery --ignore-not-found
kubectl -n $NS delete pod pgbackrest-recovery-toolbox --ignore-not-found
# Delete recovery PVCs only after confirming no data is needed.
kubectl -n $NS get pvc | grep acid-pgbackrest-recovery
```

Do not delete repository or live PVCs

The pgBackRest repository contains your backup history. The acid-postgres-ha PVCs are the live database. Never include either in a generic cleanup command.

15. RPO / RTO and Suggested Backup Schedule

Backup	Example cadence	Purpose	RPO effect	Restore effect
WAL archive	Continuous	PITR stream	Minutes/seconds depending archive latency	More WAL can increase replay time
Full	Weekly	Independent base	Anchors PITR window	Fastest dependency chain
Differential	Daily	Changed since full	Does not replace WAL	Fewer dependencies than incremental chain
Incremental	Every 4-6 hours	Changed since previous backup	Does not replace WAL	Small backup, longer dependency chain
Logical pg_dump	Daily/weekly critical DBs	Object-level recovery	Separate from physical RPO	Easy single DB/table restore

15.1 Example backup commands for automation

```
# Weekly full
pgbackrest --stanza=postgres-ha --type=full backup

# Daily differential
pgbackrest --stanza=postgres-ha --type=diff backup

# Frequent incremental
pgbackrest --stanza=postgres-ha --type=incr backup

# Always monitor archive_command and pg_stat_archiver continuously
```

15.2 Recovery drill recommendation

- Perform a scheduled restore test at least monthly.
- Validate data, roles, extensions, application login and row counts.
- Record actual restore time to measure RTO.
- Verify that the PITR target can be selected and that archived WAL exists for the required window.
- Test after major PostgreSQL, Spilo, pgBackRest, storage, IAM or operator changes.

16. Final Coverage Checklist

Requested topic / exercise	Covered	Lab section	Live-cluster rule
Logical Backup	Yes	Lab 1	Backup live; restore isolated
Physical Backup	Yes	Lab 2	Online pg_basebackup
WAL Archiving	Yes	Lab 3 + Lab 4	Validate before PITR
PITR	Yes	Lab 9	Restore isolated
pgBackRest	Yes	Lab 4 onward	Custom immutable image
Repository	Yes	Lab 4 / 10	GCS example
Compression	Yes	Lab 10	zstd example
Encryption	Yes	Lab 10	AES-256-CBC + Secret
Incremental Backup	Yes	Lab 6	Live backup
Differential Backup	Yes	Lab 7	Live backup
Retention Policies	Yes	Lab 10	Dry-run expire first
Configure pgBackRest	Yes	Lab 4	Controlled rolling change
Full Backup	Yes	Lab 5	Live backup
Restore Database / Cluster	Yes	Lab 8	Recovery target only
PITR Recovery	Yes	Lab 9	Recovery target only

References and source basis

- PostgreSQL 18 Documentation - Chapter 25, Backup and Restore; continuous archiving and PITR.
- pgBackRest User Guide and Configuration Reference - stanza, archive-push/archive-get, backup types, repository encryption, retention and restore.
- Zalando Postgres Operator documentation - custom dockerImage, sidecars/additionalVolumes, Spilo WAL-G/WAL-E backup behavior and cluster manifest parameters.
- Original supplied Day 4 manual - Kubernetes PostgreSQL 18 Backup & Disaster Recovery; live-cluster inspection, logical backup, WAL archive checks and isolated restore safety model.

End-of-Lab Success Criteria

- You can identify the current Patroni primary and confirm all members are healthy.
- You created and verified logical and physical backups from a running cluster.
- You validated WAL archiving and can explain why an unbroken WAL chain is required for PITR.
- You configured pgBackRest using an immutable image and durable repository.
- You completed full, incremental and differential backup exercises.
- You configured compression, encryption and retention.
- You restored a physical backup only to isolated recovery storage.
- You completed PITR to a timestamp before an intentional DELETE and verified the recovered rows.

Production sign-off

A backup is not considered successful until it has been restored and validated. Schedule recurring recovery drills and monitor archive failures, repository capacity, backup age and restore duration.